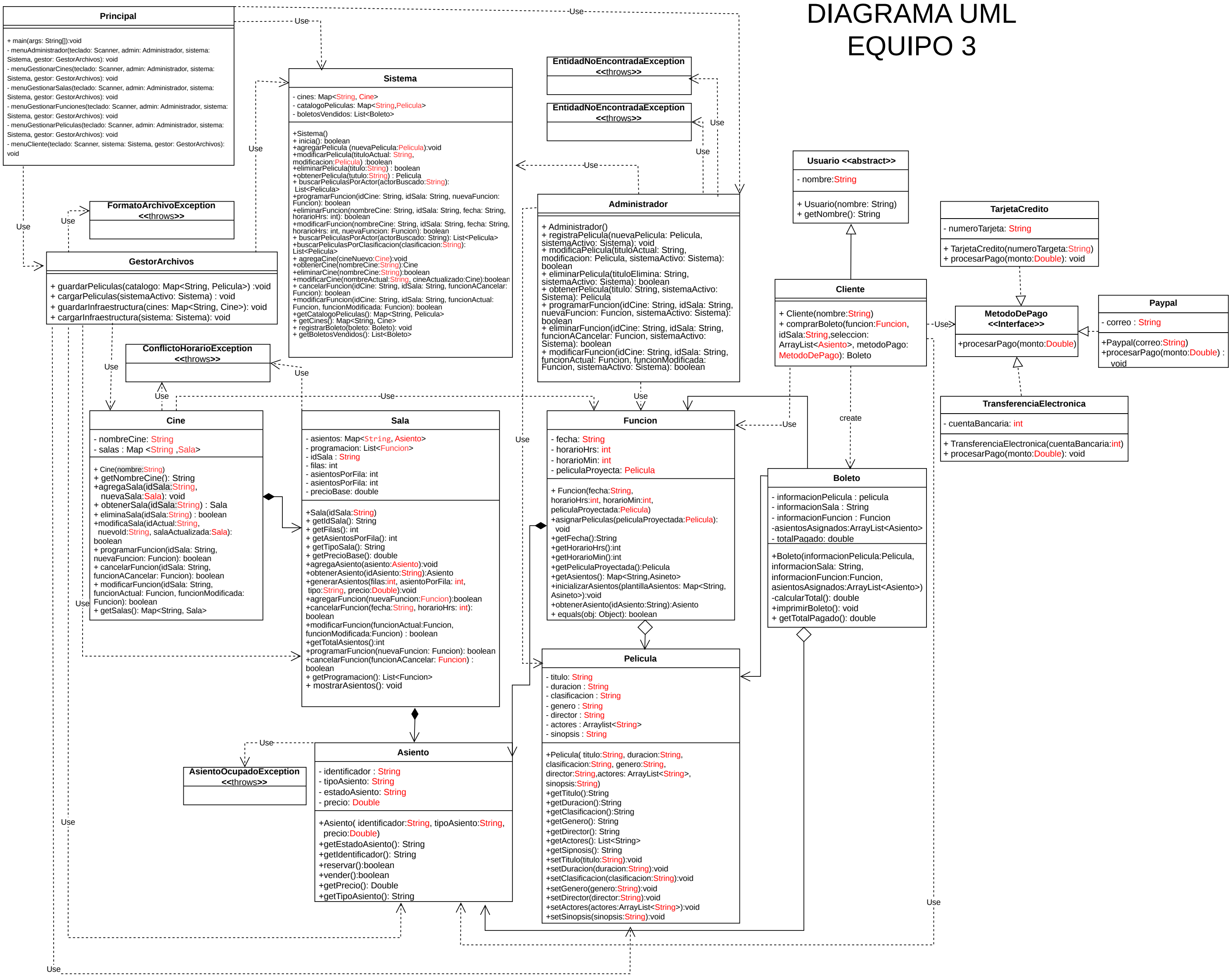
DIAGRAMA UML
EQUIPO 3


Principal
+ main(args: String[]):void
- menuAdministrador(teclado: Scanner, admin: Administrador, sistema: Sistema, gestor: GestorArchivos): void
- menuGestionarCines(teclado: Scanner, admin: Administrador, sistema: Sistema, gestor: GestorArchivos): void
- menuGestionarSalas(teclado: Scanner, admin: Administrador, sistema: Sistema, gestor: GestorArchivos): void
- menuGestionarFunciones(teclado: Scanner, admin: Administrador, sistema: Sistema, gestor: GestorArchivos): void
- menuGestionarPeliculas(teclado: Scanner, admin: Administrador, sistema: Sistema, gestor: GestorArchivos): void
- menuCliente(teclado: Scanner, sistema: Sistema, gestor: GestorArchivos): void
Use
Use
Sistema
- cines: Map<String, Cine>
- catalogoPeliculas: Map<String,Pelicula>
- boletosVendidos: List<Boleto>
+Sistema()
+ inicia(): boolean
+agregarPelicula (nuevaPelicula:Pelicula):void
+modificarPelicula(tituloActual: String, modificacion:Pelicula) :boolean
+eliminarPelicula(titulo:String) : boolean
+obtenerPelicula(tutulo:String) : Pelicula
+ buscarPeliculasPorActor(actorBuscado:String): List<Pelicula>
+programarFuncion(idCine: String, idSala: String, nuevaFuncion: Funcion): boolean
+eliminarFuncion(nombreCine: String, idSala: String, fecha: String, horarioHrs: int): boolean
+modificarFuncion(nombreCine: String, idSala: String, fecha: String, horarioHrs: int, nuevaFuncion: Funcion): boolean
+ buscarPeliculasPorActor(actorBuscado: String): List<Pelicula>
+buscarPeliculasPorClasificacion(clasificacion:String): List<Pelicula>
+ agregaCine(cineNuevo:Cine):void
+obtenerCine(nombreCine:String):Cine
+eliminarCine(nombreCine:String):boolean
+modificarCine(nombreActual:String, cineActualizado:Cine):boolean
+ cancelarFuncion(idCine: String, idSala: String, funcionACancelar: Funcion): boolean
+modificarFuncion(idCine: String, idSala: String, funcionActual: Funcion, funcionModificada: Funcion): boolean
+getCatalogoPeliculas(): Map<String, Pelicula>
+ getCines(): Map<String, Cine>
+ registrarBoleto(boleto: Boleto): void
+ getBoletosVendidos(): List<Boleto>
Use
EntidadNoEncontradaException
<<throws>>
EntidadNoEncontradaException
<<throws>>
Use
Use
Use
Administrador
+ Administrador()
+ registraPelicula(nuevaPelicula: Pelicula, sistemaActivo: Sistema): void
+ modificaPelicula(tituloActual: String, modificacion: Pelicula, sistemaActivo: Sistema): boolean
+ eliminarPelicula(tituloElimina: String, sistemaActivo: Sistema): boolean
+ obtenerPelicula(titulo: String, sistemaActivo: Sistema): Pelicula
+ programarFuncion(idCine: String, idSala: String, nuevaFuncion: Funcion, sistemaActivo: Sistema): boolean
+ eliminarFuncion(idCine: String, idSala: String, funcionACancelar: Funcion, sistemaActivo: Sistema): boolean
+ modificarFuncion(idCine: String, idSala: String, funcionActual: Funcion, funcionModificada: Funcion, sistemaActivo: Sistema): boolean
Usuario <<abstract>>
- nombre:String
+ Usuario(nombre: String)
+ getNombre(): String
TarjetaCredito
- numeroTarjeta: String
+ TarjetaCredito(numeroTargeta:String)
+ procesarPago(monto:Double): void
Use
FormatoArchivoException
<<throws>>
Use
GestorArchivos
+ guardarPeliculas(catalogo: Map<String, Pelicula>) :void
+ cargarPeliculas(sistemaActivo: Sistema) : void
+ guardarInfraestructura(cines: Map<String, Cine>): void
+ cargarInfraestructura(sistema: Sistema): void
Cliente
+ Cliente(nombre:String)
+ comprarBoleto(funcion:Funcion, idSala:String,seleccion: ArrayList<Asiento>, metodoPago: MetodoDePago): Boleto
Use
MetodoDePago
<<Interface>>
+procesarPago(monto:Double)
Paypal
- correo : String
+Paypal(correo:String)
+procesarPago(monto:Double) : void
ConflictoHorarioException
<<throws>>
Use
Use
Use
Use
Use
Use
create
Use
TransferenciaElectronica
- cuentaBancaria: int
+ TransferenciaElectronica(cuentaBancaria:int)
+ procesarPago(monto:Double): void
Cine
- nombreCine: String
- salas : Map <String ,Sala>
+ Cine(nombre:String)
+ getNombreCine(): String
+agregaSala(idSala:String, nuevaSala:Sala): void
+ obtenerSala(idSala:String) : Sala
+ eliminaSala(idSala:String) : boolean
+modificaSala(idActual:String, nuevoId:String, salaActualizada:Sala): boolean
+ programarFuncion(idSala: String, nuevaFuncion: Funcion): boolean
+ cancelarFuncion(idSala: String, funcionACancelar: Funcion): boolean
+ modificarFuncion(idSala: String, funcionActual: Funcion, funcionModificada: Funcion): boolean
+ getSalas(): Map<String, Sala>
Use
Sala
- asientos: Map<String, Asiento>
- programacion: List<Funcion>
- idSala : String
- filas: int
- asientosPorFila: int
- asientosPorFila: int
- precioBase: double
+Sala(idSala:String)
+ getIdSala(): String
+ getFilas(): int
+ getAsientosPorFila(): int
+ getTipoSala(): String
+ getPrecioBase(): double
+agregaAsiento(asiento:Asiento):void
+obtenerAsiento(idAsiento:String):Asiento
+generarAsientos(filas:int, asientoPorFila: int, tipo:String, precio:Double):void
+agregarFuncion(nuevaFuncion:Funcion):boolean
+cancelarFuncion(fecha:String, horarioHrs: int): boolean
+modificarFuncion(funcionActual:Funcion, funcionModificada:Funcion) : boolean
+getTotalAsientos():int
+programarFuncion(nuevaFuncion: Funcion): boolean
+cancelarFuncion(funcionACancelar: Funcion) : boolean
+ getProgramacion(): List<Funcion>
+ mostrarAsientos(): void
Use
Funcion
- fecha: String
- horarioHrs: int
- horarioMin: int
- peliculaProyecta: Pelicula
+ Funcion(fecha:String, horarioHrs:int, horarioMin:int, peliculaProyectada:Pelicula)
+asignarPeliculas(peliculaProyectada:Pelicula): void
+getFecha():String
+getHorarioHrs():int
+getHorarioMin():int
+getPeliculaProyectada():Pelicula
+getAsientos(): Map<String,Asineto>
+inicializarAsientos(plantillaAsientos: Map<String, Asineto>):void
+obtenerAsiento(idAsiento:String):Asiento
+ equals(obj: Object): boolean
Boleto
- informacionPelicula : pelicula
- informacionSala : String
- informacionFuncion : Funcion
-asientosAsignados:ArrayList<Asiento>
- totalPagado: double
+Boleto(informacionPelicula:Pelicula, informacionSala: String, informacionFuncion:Funcion, asientosAsignados:ArrayList<Asiento>)
-calcularTotal(): double
+imprimirBoleto(): void
+ getTotalPagado(): double
Pelicula
- titulo: String
- duracion : String
- clasificacion : String
- genero : String
- director : String
- actores : Arraylist<String>
- sinopsis : String
+Pelicula(titulo:String, duracion:String, clasificacion:String, genero:String, director:String,actores: ArrayList<String>, sinopsis:String)
+getTitulo():String
+getDuracion():String
+getClasificacion():String
+getGenero(): String
+getDirector(): String
+getActores(): List<String>
+getSipnosis(): String
+setTitulo(titulo:String):void
+setDuracion(duracion:String):void
+setClasificacion(clasificacion:String):void
+setGenero(genero:String):void
+setDirector(director:String):void
+setActores(actores:ArrayList<String>):void
+setSinopsis(sinopsis:String):void
Use
AsientoOcupadoException
<<throws>>
Asiento
- identificador : String
- tipoAsiento: String
- estadoAsiento: String
- precio: Double
+Asiento(identificador:String, tipoAsiento:String, precio:Double)
+getEstadoAsiento(): String
+getIdentificador(): String
+reservar():boolean
+vender():boolean
+getPrecio(): Double
+getTipoAsiento(): String
Use
Use
Use